

Cover Letter Templates — All Versions

Yuanjun (Johnny) Dai

SHARED SECTIONS (same across all versions)

Paragraph 2 — Career Narrative

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

Paragraph 4 — Why I Fit + Closing

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 0: OVERALL (General Application)

Dear Hiring Team,

I am a PhD candidate in Computer Science at Case Western Reserve University graduating in December 2026, and I am actively seeking full-time roles in AI infrastructure, ML systems, or research software engineering. My research focuses on the observability, optimization, and security of distributed ML and LLM training and inference systems, and I have published seven papers at IEEE, ACM, and Springer on these topics. I am writing to express my interest in joining your team and contributing to the work you are doing at the intersection of systems engineering and AI.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

The work I am most proud of spans the full stack. I designed DYNAMIX, a reinforcement learning-based batch scheduler that achieved a 46% wall-clock training time reduction across 8–64 A100 GPUs at national HPC facilities. I built PSketch and eHashPipe, two in-kernel eBPF monitoring frameworks that bring microsecond-resolution observability to distributed ML workloads. I designed and deployed a production LLM serving platform on Kubernetes with full GPU observability, GitOps CI/CD, and hardware MIG partitioning — validated under extreme stress on Lambda Labs A100 hardware.

Across all of these, the common thread is building systems that are reliable, observable, and usable by people who depend on them.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 1: RSE (Research Software Engineer)

Dear Hiring Team,

I am applying for the Research Software Engineer position. I am a PhD candidate in Computer Science at CWRU graduating in December 2026, with direct hands-on experience running large-scale experiments on national HPC resources including the Ohio Supercomputer Center, NSF FABRIC testbed, and Lambda Labs. I understand what it means to work in an HPC environment — not as background infrastructure, but as the environment where the research actually happens. I want to bring that experience to bear in supporting faculty and researchers who depend on systems like HiPerGator to do their work.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too

fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

My research is relevant to this role in a direct way. I ran distributed ML training jobs across 8–64 A100 GPUs on OSC using SLURM, debugging NCCL communication failures and tuning multi-node job configurations. I built in-kernel eBPF monitoring frameworks that give 10–14x finer temporal resolution than existing tools — exactly the kind of observability that HPC operators need when something goes wrong mid-job. And through my clinical NLP collaboration with medical researchers, I developed a working instinct for what it means to translate a domain expert's question into something a computing system can actually run. For me, success in this role is not measured by what I build — it is measured by whether the researchers I support can do their work more effectively.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 2: SWE Infrastructure

Dear Hiring Team,

I am applying for the Software Engineering — Infrastructure position. I am a PhD candidate in Computer Science at CWRU graduating in December 2026, with a background that runs from low-level ASIC firmware at Cisco and Broadcom to production Kubernetes deployments and distributed systems research. Infrastructure engineering is not a pivot for me — it is what I have been doing across industry and academia for the past decade, and I want to continue doing it at a company operating at your scale.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

The infrastructure work I have done spans the full stack. I independently designed and deployed a self-hosted LLM serving platform on Kubernetes — three-tier architecture, GitOps CI/CD via ArgoCD and GitHub Actions, GPU time-slicing and MIG partitioning across hardware tiers, and a 4-layer observability stack wiring DCGM into Prometheus and Grafana. I built Scotch, a distributed OvS-based overlay for SDN control-plane scalability published in ACM TOIT, that reduces controller congestion by over 70% on a 34-node topology. I also built in-kernel eBPF monitoring systems that operate at microsecond resolution without modifying application code. These are not research prototypes — they are engineered systems designed to be reliable and extensible.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 3: AI Infra Observability

Dear Hiring Team,

I am applying for the AI Infrastructure Observability Engineer position. My PhD research is almost entirely about observability — not as a feature, but as a fundamental requirement for operating distributed ML systems reliably. I built two published in-kernel monitoring frameworks from scratch, designed and deployed a full-stack GPU observability pipeline for a production LLM serving platform, and spent years thinking about what it actually means to understand what a large-scale AI system is doing at runtime. That is exactly the problem I want to work on.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

The observability work I have done is concrete. PSketch is a three-tier in-kernel monitoring system built entirely in eBPF — exact per-flow tracking for priority traffic via a hashmap, elephant flow identification via a heavy flow table with voting-based collision resolution, and mice flow estimation via a 3-layer Count-Min Sketch — achieving 96% Top-K accuracy at 10 Gbps with under 1.1% throughput overhead. eHashPipe extends this to system resources, delivering 10–14x finer temporal resolution than top or htop. For my LLM serving platform, I built a 4-layer GPU observability stack — DCGM Exporter for GPU hardware metrics, vLLM business metrics, FastAPI gateway counters and latency histograms, and Kubernetes cluster health — all wired into Prometheus and Grafana with auto-imported dashboards. I have also thought deeply about what comes

next: the real unsolved problem is not collecting observability data, it is diagnosing what it means. I believe agentic observability — combining telemetry with LLM reasoning — is where this field is headed.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 4: ML Systems

Dear Hiring Team,

I am applying for the ML Systems Engineer position. Distributed ML training optimization has been the central focus of my PhD research — not as an application domain, but as a systems problem in its own right. I have designed and validated systems that make large-scale ML training meaningfully faster and more reliable, published the results at IEEE IPDPS, and I want to continue doing this kind of work at a company where it has direct product impact.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU

specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

DYNAMIX is the clearest example of what I can do: a PPO-based reinforcement learning scheduler that monitors hardware signals — CPU, memory, network bandwidth, and retransmission rate via custom eBPF probes — alongside statistical learning signals to dynamically adapt batch size throughout training. The key insight was that batch size is simultaneously a hardware efficiency lever and a model generalization lever. By co-scaling the learning rate via the Linear Scaling Rule and validating across both PyTorch DDP and BytePS architectures on 8–64 A100 GPUs at national HPC facilities, I achieved a 46% wall-clock training time reduction and a 6% accuracy improvement. PRISM complements this at the transport layer: a library that hooks into PyTorch DDP's AllReduce at bucket-ready events, providing priority-based selective reliability for gradient traffic without modifying training code, achieving 8–21% throughput gain over TCP under packet loss.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 5: AI Infrastructure

Dear Hiring Team,

I am applying for the AI Infrastructure Engineer position. My PhD research has been entirely focused on the systems that make large-scale AI training and inference work — scheduling, transport, observability, and deployment. I have published seven papers on these topics, run experiments on 64 A100 GPUs at national HPC facilities, and deployed production LLM serving infrastructure. This is

not adjacent to what I want to do — it is exactly what I want to do, and I want to do it at a company where the infrastructure I build serves models and users at real scale.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

The infrastructure work I have done spans training, transport, and serving. On the training side, DYNAMIX achieved a 46% end-to-end speedup on 64 A100 GPUs by using RL-based adaptive batch scheduling driven by eBPF hardware telemetry. PRISM improved distributed training throughput by 8–21% by redesigning gradient transport with priority-based selective reliability. On the serving side, I built a production Kubernetes LLM inference platform supporting GPU time-slicing, MIG partitioning, custom-metrics HPA autoscaling, and 4-layer GPU observability — validated under extreme load on Lambda Labs A100 hardware. I also fine-tuned PubMedBERT and Qwen-7B using NeMo, Megatron-LM, and FSDP ZeRO-3, which gave me direct experience with the full training pipeline from data to deployed model.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

VERSION 6: MLE (Machine Learning Engineer)

Dear Hiring Team,

I am applying for the Machine Learning Engineer position. I have worked across the full ML lifecycle — from building and fine-tuning large language models on multi-GPU clusters to designing the serving infrastructure that makes them usable in production. I am not a researcher looking to transition into engineering: I have done both, and I want a role where both matter.

My career path looks nonlinear on paper, but there is a consistent logic running through it. I started in systems engineering at Cisco and Broadcom, working on NPU/ASIC features for 100 Gbps core routers and low-level RTOS pipelines. I then moved deliberately into product management at Midea and Xiaomi — not because I wanted to leave engineering, but because I wanted to understand how technology creates value when it reaches real users. The product I led at Midea won the Red Dot Design Award and shipped commercially. But working in that role made the bottleneck clear: the limiting factor was almost never the features — it was the infrastructure underneath. Training pipelines were too slow. Deployment was too fragile. Nothing was observable. I returned to research at CWRU specifically to go deeper on those problems, and that has been the focus of my PhD ever since.

The applied ML work I am most proud of includes two complementary projects. For clinical NLP, I built an end-to-end multi-label text classifier across 115 highly imbalanced labels — applying inverse-frequency weighted loss and per-label threshold calibration to improve long-tail performance, and fine-tuning both PubMedBERT and Qwen-7B using NeMo, Megatron-LM, and FSDP ZeRO-3 within GPU memory constraints. I also used few-shot prompting with GPT-4o to build the labeled training dataset, which required careful prompt engineering to mitigate hallucination on rare medical codes. On the agentic AI side, I built a multi-agent LangGraph system for automated

job discovery and resume optimization — with capability-aware multi-model routing (DeepSeek for high-volume scoring, Claude Opus for quality-critical tailoring) and a 6-dimension LLM matching framework designed via structured prompt engineering. Both projects show the same thing: I can take an ambiguous problem, design the right ML approach, engineer the system around it, and ship something that works.

What I bring to this role is uncommon: the systems instinct of someone who has debugged kernel-level failures, the product judgment of someone who has shipped to real users, and the research rigor of someone who has published the results. I am ready to apply all three to real problems at scale.

Thank you for your consideration.

Best regards, Yuanjun (Johnny) Dai yxd429@case.edu | 216-269-2394
johnny-dai-git.github.io/portfolio

QUICK REFERENCE — Paragraph 1 & 3 by Version

| Version | P1 Focus | P3 Projects | |---|---|---| | Overall | General
interest, breadth | DYNAMIX + PSketch + LLM Serving | | RSE | HPC
experience + support mission | OSC/SLURM + eBPF monitoring +
researcher collaboration | | SWE Infra | 10yr systems background |
LLM Serving (K8s) + Scotch + eBPF | | Observability | PhD =
observability research | PSketch/eHashPipe (detail) + LLM monitoring
stack | | ML Systems | Training optimization focus | DYNAMIX
(detail) + PRISM | | AI Infra | Full AI infra stack | DYNAMIX + PRISM
+ LLM Serving + fine-tuning | | MLE | Applied ML + ship products |
Clinical LLM (detail) + Job Agent |